OWASP Juice Shop Web security analysis

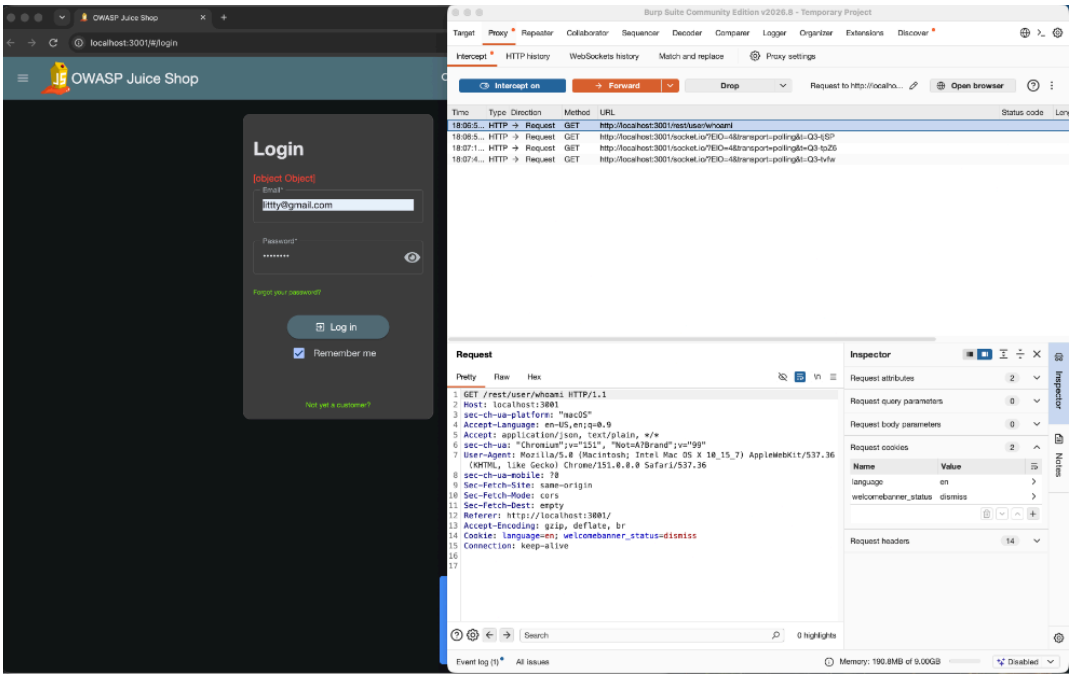
Dhruv Guda

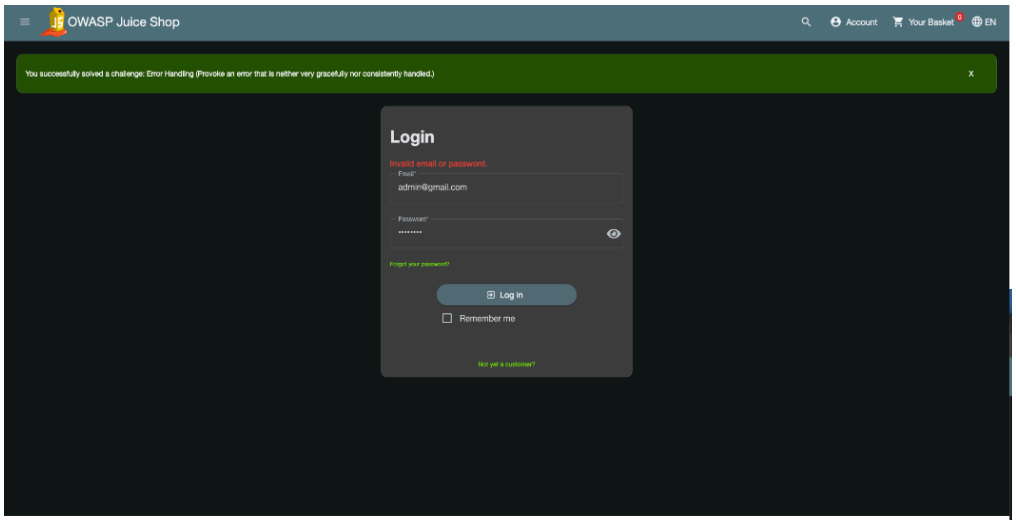
10/3/2026

Introduction

This report examines common web application vulnerabilities using OWASP Juice Shop and a local login lab. It analyzes SQL injection, DOM-based cross-site scripting, exposed credentials, and error handling, explaining their security impact and recommended mitigations. The focus is on understanding insecure design through controlled educational testing.

Vulnerability #1: Inconsistent Error Handling





Error:

[object Object]

The message:

You successfully solved a challenge:

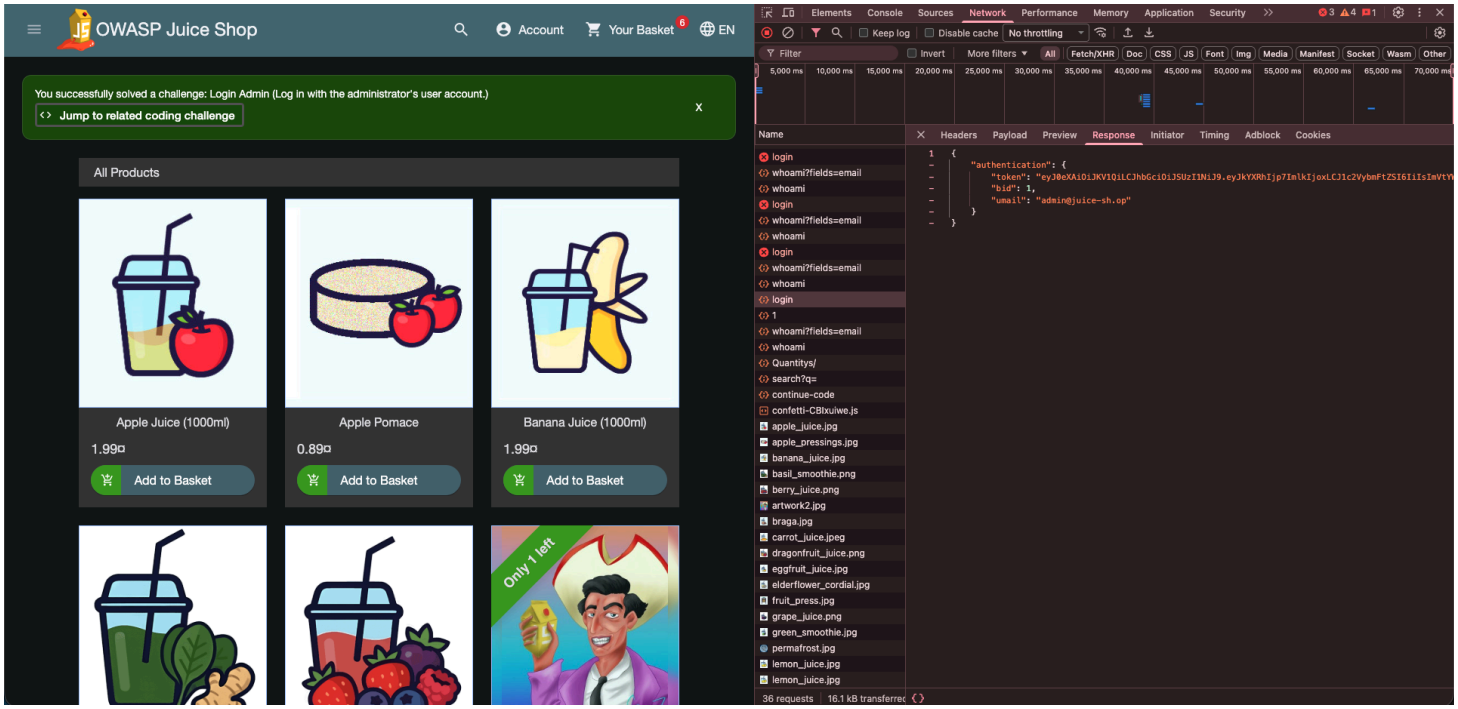
Error Handling (Provoke an error that is neither very gracefully nor consistently handled.)

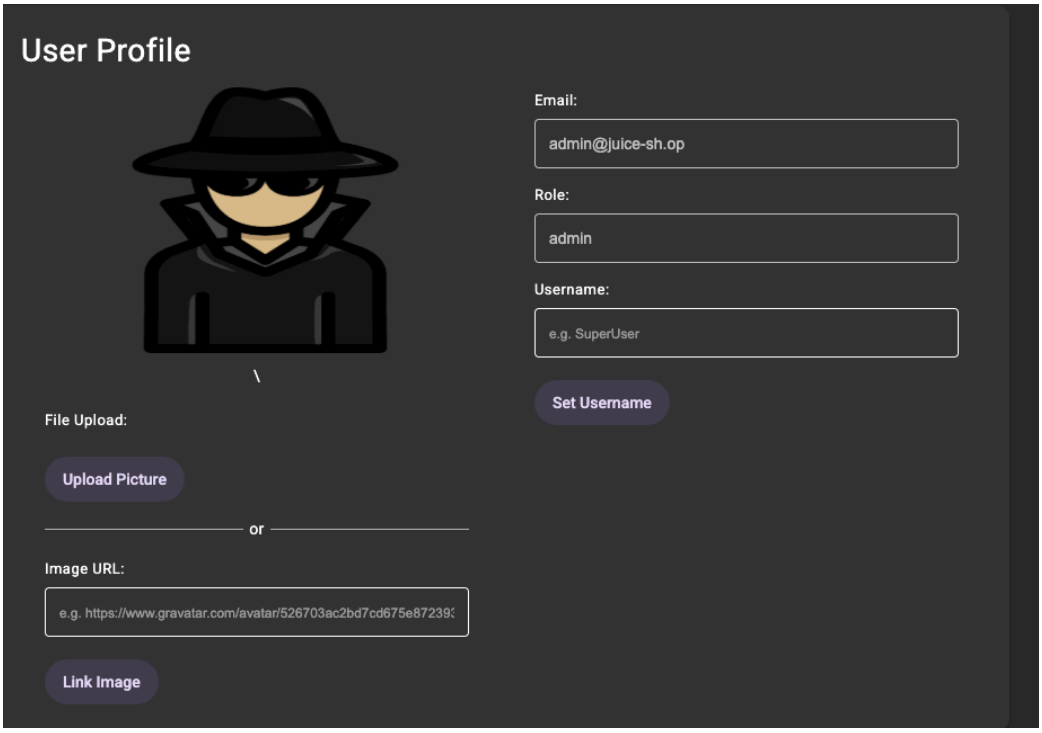
The how:

Used Burp Suite to intercept the connection between the application and the server checking for the correct email and password.

Intentionally malformed request body: I changed the query for “whoami” to…

```
None
{
"email":"1@"=1"gmail.com",
"Password":"password"
}
```





Mitigation

Malformed requests should receive consistent, understandable error responses without exposing internal implementation details. Detailed diagnostics should be recorded in protected server-side logs, with sensitive values excluded. These controls are proposed mitigations; their implementation was not verified in this analysis.

Vulnerability #2: SQL Injection Authentication Bypass

The message:

You successfully solved a challenge:

The how:

I used SQL injection in order to change the SQLite query to set it to always true.

```
None
{
  "authentication": {
```

```
    "token":"SOME LARGE HEX VALUE",
    "bid": 1,
    "umail": "admin@juice-sh.op"
  }
}
```

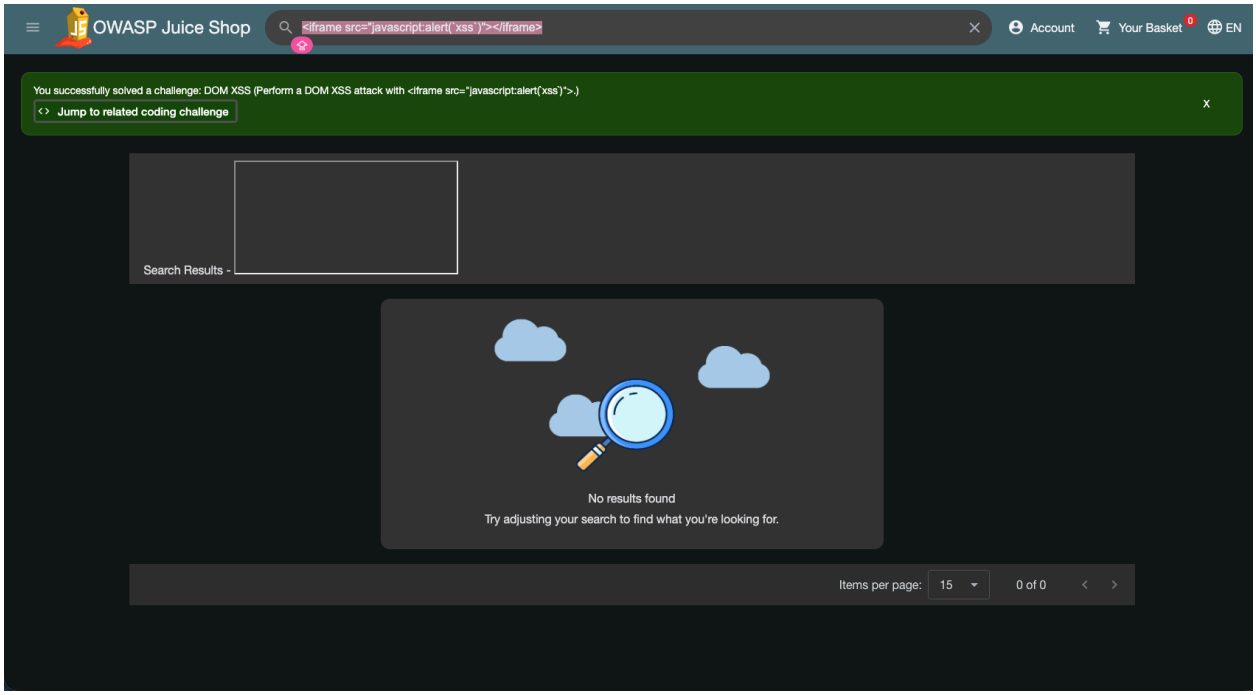

Mitigation & Explanation:

One way to mitigate SQL injection in this situation is by using prepared statements.

Using prepared statements with parameterized queries allows the query to search for exactly what was entered.

Prepared statements with bound parameters keep submitted values separate from SQL syntax. A string containing quotation marks or SQL operators is treated as a value rather than as an instruction that changes the query. This addresses the demonstrated injection mechanism; it does not, by itself, provide secure password storage or a complete authentication system (OWASP Foundation, n.d.-f).

Vulnerability #3: DOM-Based XSS



The message:

You successfully solved a challenge: DOM XSS (Perform a DOM XSS attack with <iframe src="javascript:alert(`xss`)">.)

The how:

To test if the *Cross-Site Scripting* method works, I tried <h1> OWASP and saw that the output was much larger text compared to the normal OWASP search entry and that it suggested HTML interpretation. The enlarged heading indicated HTML interpretation but did not establish JavaScript execution. I then tested the various payloads, such as the payload below.

<iframe src="javascript:alert(`xss`)"></iframe>

Mitigation & Explanation:

To mitigate DOM cross-site scripting, make sure the application treats user input strictly as plain text instead of executable code.

When developers put a search query directly on the page, they should use safe methods like textContent rather than innerHTML. This ensures that if someone enters an iframe tag, the browser just displays it as normal text rather than running the malicious alert.

By relying on these built-in features, developers stop the browser from confusing user input with actual website code.

Vulnerability #4: Exposed Credentials

The message:

You successfully solved a challenge: Exposed Credentials (a developer was careless with hardcoding unused but still valid credentials for a testing account on the client-side.)

The how:

To determine if there are any hardcoded passwords in the client-side dev files I initially went through the files trying to find “login” to see if there was a certain function that contained the credentials.

After that I tested “username” to see if there was a variable name with a section of it called username.

This is when I found the section with the hardcoded credentials…

```
None
testingUsername = `testing@juice-sh.op`;
testingPassword = `IamUsedForTesting`;
```

Using these credentials, I was able to get into the testing account.

Mitigation & Explanation:

Valid credentials must not be embedded in browser-delivered code. If real credentials are exposed, revoke or rotate them and remove them from distributed files and relevant repository history. Server-side secrets should use controlled secret storage, supported by automated secret scanning and code review.

Example of secure password handling:

Passwords should be stored using a purpose-built password-hashing algorithm such as Argon2id, with unique salts and appropriate work parameters. This makes offline guessing more expensive if password hashes are stolen; it does not make passwords impossible to recover. MD5 is unsuitable for production password storage.

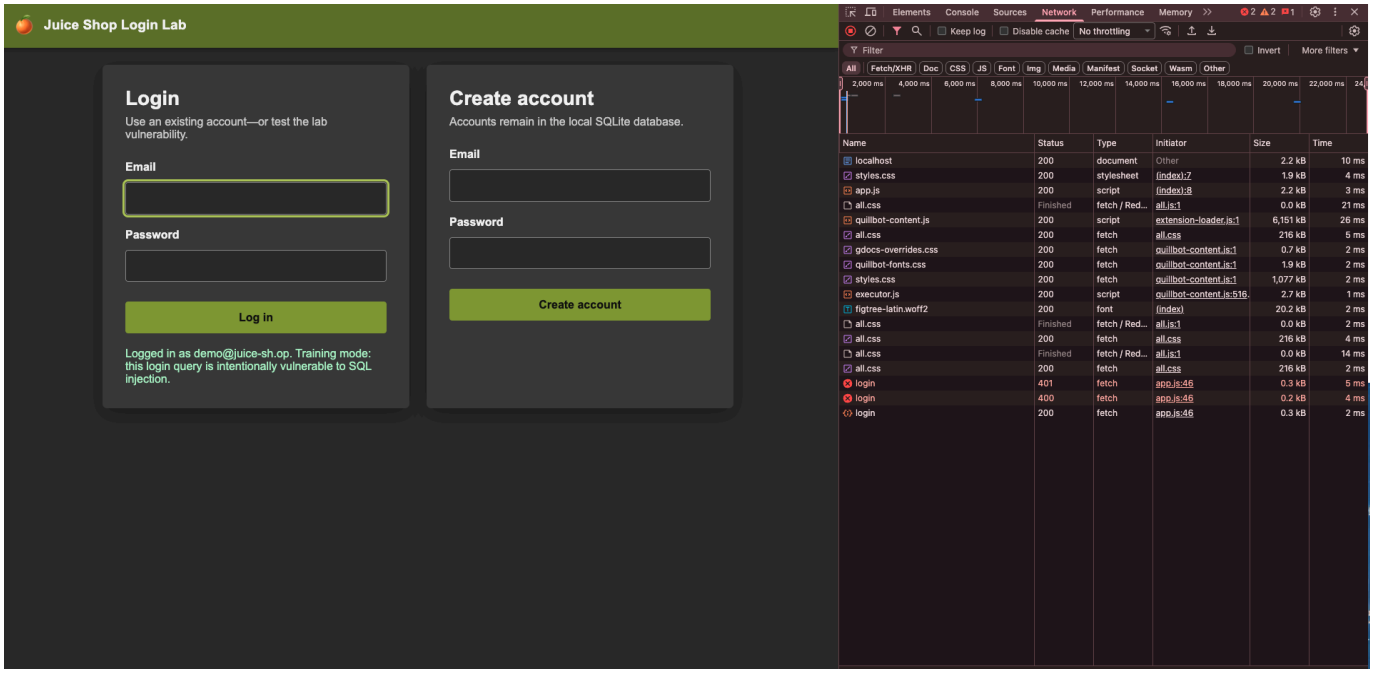
Part 2: Local Registration and Login Lab

I built a local login lab using HTML, CSS, JavaScript, Node.js, and SQLite, with registration and persistent test accounts. Registration checks for an email containing an @ symbol and a password of at least eight characters; login requires only nonempty input. The lab deliberately uses insecure MD5 password hashing and a SQL-injection-vulnerable login query, allowing an attacker to bypass normal credential checks. These weaknesses support controlled educational testing; a successful login response does not establish an authenticated session.

GitHub repository:

github.com/d-guda/CS477-Login-Form

Part 3: Exploit a Vulnerability



Looking at the code, the first vulnerability that came to mind was SQL injection.

I initially tried a random email and password to see the 401 Unauthorized. I then tried the SQL injection ‘ OR 1=1 – @ and this worked.

Proposed SQL-Injection Remediation

One fix is to replace the dynamically constructed SQL query with a parameterized query. Instead of inserting the submitted email directly into the SQL statement, the server should use a placeholder:

```
None
const query = database.prepare(
  'SELECT id, email FROM users WHERE email = ? AND password = ? LIMIT 1'
);

const user = query.get(email, hashPassword(password));
```

Parameterization prevents the demonstrated SQL injection by treating submitted values as data rather than SQL syntax. This proposed change does not address the lab’s weak MD5 password hashing or provide a complete authentication system. Password storage would also need an appropriate password-hashing algorithm, such as Argon2id.

References

OWASP Foundation. (n.d.-a). *DOM based XSS prevention cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/DOM_based_XSS_Prevention_Cheat_Sheet.html

OWASP Foundation. (n.d.-b). *Error handling cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Error_Handling_Cheat_Sheet.html

OWASP Foundation. (n.d.-c). *OWASP Juice Shop*. https://owasp.org/projects/juice-shop

OWASP Foundation. (n.d.-d). *Password storage cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

OWASP Foundation. (n.d.-e). *Secrets management cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html

OWASP Foundation. (n.d.-f). *SQL injection prevention cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html